

Guide to the Short Python Exercises

k-NN, K-Means and Linear Regression in a few lines of NumPy
Companion to `Exercises_Short_Python.ipynb` (Exercises 1 to 12)

How to use this guide. Read the *key material* section before starting. Then, for each question, try on your own first. If you are stuck for more than a few minutes, open the hints in order: **Hint 1** points you in the right direction, **Hint 2** gives the method, **Hint 3** takes you to the last step. The hints never give the final numbers: the point is that you produce them. The *common mistakes* boxes list what goes wrong most often; read them even when you think you are done.

Key material: NumPy patterns used throughout

Vectorised arithmetic

Most exercises are one or two lines once you think in whole arrays rather than loops.

```
P - q                # (n, p) minus (p,): q is subtracted from
    every row
((P - q) ** 2).sum(axis=1) # row-wise sum -> (n,) squared distances
np.abs(P - q).sum(axis=1) # Manhattan distances
np.linalg.norm(M, axis=1) # Euclidean norm of each row
x[:, None] - c[None, :]  # (n, K) table of all point-centre differences
M.argmax(axis=1)         # index of the smallest value in each row
np.all(cond, axis=1)      # True where every column of a row is True
```

Counting and voting

```
np.argsort(d, kind="stable")[:k] # indices of the k smallest values
np.bincount(labels, minlength=2) # how many 0s, how many 1s
np.bincount(labels, weights=w)   # sum of w for each label value
values, counts = np.unique(v, return_counts=True) # works for strings
too
values[counts.argmax()]          # the most frequent value
```

`kind="stable"` keeps equal distances in their original order, so results are reproducible when there are ties.

Linear algebra

```
np.column_stack([np.ones_like(x), x]) # design matrix with an intercept
column
A.T @ A                               # matrix product
np.linalg.solve(M, v)                 # solves M theta = v (better than
inv)
np.linalg.matrix_rank(A), np.linalg.det(M), np.eye(3)
np.polyfit(x, y, 1)                   # returns [slope, intercept]
```

How the CHECK cells work

Each CHECK compares your result with an independent reference: `scikit-learn`, `np.polyfit`, `np.linalg.lstsq`, or a mathematical property (residuals sum to zero,

probabilities sum to one, inertia never increases). If a CHECK fails, print the intermediate arrays and compare them with the pen-and-paper exercise of the same number: the data are identical.

Part I k -Nearest Neighbours

Exercise 1 Classifying a point

euclidean and manhattan

Hint 1 · Nudge Both are one line. The only difference is square-then-sum-then-root versus absolute-value-then-sum.

Hint 2 · Method $P - q$ broadcasts q over the rows. Sum with `axis=1` to get one number per point.

knn_vote

Hint 1 · Nudge Three steps: distances with the `dist` function passed in, indices of the k smallest, then a (possibly weighted) count per class.

Hint 2 · Method Let the weights be `np.ones(k)` for a plain vote and `1 / d[idx]` for a weighted vote. Then a single call covers both cases: `np.bincount(labels[idx], weights=w, minlength=...)`.

Hint 3 · Almost there Return `totals.argmax()`. A function can be passed as an argument like any other object: `knn_vote(P, labels, q, 3, manhattan)`.

Question 1 (written)

Hint 1 · Nudge Print the sorted table (the solution layout is: point, class, d^2 , d , d_1) and watch which points enter as k grows.

Hint 2 · Method For $k = 7$: how many training points are there? Which points are the neighbours of *any* query?

Common mistakes

- `np.sort(d)[:k]` gives the smallest distances, not the points: use `argsort`.
- Weighted vote with $d = 0$ (query equal to a training point) divides by zero. It does not happen here, but real code must handle it (scikit-learn gives such points all the weight).

Exercise 2 Leave-one-out

Hint 1 · Nudge A loop over i . Inside: a boolean mask that removes point i , the distances from x_i to the other points, the k nearest labels, a vote.

Hint 2 · Method `keep = np.arange(len(x)) != i` is True everywhere except at i . `x[keep]` and `y[keep]` are the remaining points. Labels are strings here, so vote with `np.unique(..., return_counts=True)` rather than `bincount`.

Hint 3 · Almost there Accumulate mistakes `+= (prediction != y[i])` (a boolean counts as 0 or 1), and return `mistakes / len(x)`.

Question 2 (written)

Hint 1 · Nudge For $k = 7$, how many points vote, and how many A and B are left when you remove an A? When you remove a B?

Hint 2 · Method The dataset has 4 of each class. Removing a point changes the balance of the remaining 7 *against* that point.

Exercise 3 Scaling

Hint 1 · Nudge Stack the two candidates in one array of shape $(2, 2)$ so that each computation is one line.

Hint 2 · Method Raw: `np.linalg.norm(cands - q_c, axis=1)`. Scaled: divide the differences by s before taking the norm. Thousands: divide by `[1, 1000]`.

Hint 3 · Almost there The CHECK tells you which customer must be nearest in each case: if your result disagrees, you have probably divided the wrong column.

Question 3 (written)

Hint 2 · Method Compute $\sqrt{25^2 + 1000^2} - 1000$. Why is it so small? Use $\sqrt{a^2 + \varepsilon^2} \approx a + \varepsilon^2/(2a)$ for $\varepsilon \ll a$.

Exercise 4 Curse of dimensionality

Hint 1 · Nudge `edge` is the solution of $e^d = p$: one line with `**`.

Hint 2 · Method A point u is inside the central cube of side e when every coordinate satisfies $|u_j - 0.5| \leq e/2$. Test this for all rows at once and take the mean of the resulting booleans.

Hint 3 · Almost there `np.all(np.abs(U - 0.5) <= e / 2, axis=1).mean()`. Create `rng` once before the loop.

Mathematical insight · why the Monte Carlo fraction is close to p

Each coordinate falls in the interval independently with probability e , so all d coordinates do with probability $e^d = p$. The count of points inside is $\text{binomial}(N, p)$, with standard deviation $\sqrt{Np(1-p)}$; for $N = 200\,000$ and $p = 0.01$ the fraction has a standard deviation of about 0.0002. That is why the CHECK can use a tolerance of 0.002.

Part II *K*-Means

Exercise 5 Lloyd in 1-D

Hint 1 · Nudge Structure: a loop; inside, assign; if the assignment did not change, stop; otherwise update the centres.

Hint 2 · Method Assign: `np.abs(x[:, None] - c[None, :]).argmin(axis=1)` gives, for each point, the index of the nearest centre. Update: `np.array([x[labels == j].mean() for j in range(len(c))])`.

Hint 3 · Almost there Compare the new labels with the previous ones using `(new == labels).all()`. On the first pass there are no previous labels: start with `labels = None`. The inertia is `((x - c[labels]) ** 2).sum()`.

Common mistakes

- Updating the centres before checking convergence: you then compare labels from different centres.
- An empty cluster makes `mean()` return `nan`. It cannot happen with these initialisations; Exercise 7 shows one way to guard against it.

Exercise 6 *k*-means++

Hint 2 · Method $D(x)^2 = (x - 1)^2$ because there is only one centre. Normalise by the sum.

Hint 3 · Almost there `rng.choice(x5, size=20_000, p=probs)` draws with those probabilities; `(draws == 25).mean()` is the frequency.

Exercise 7 Inertia never increases

Hint 1 · Nudge Same two steps as in Exercise 5, but in 2-D, with one inertia computation after each step.

Hint 2 · Method 2-D assignment: `((Xb[:, None, :] - C[None, :, :]) ** 2).sum(axis=2).argmin(axis=1)`, a table of shape (points, centres). Update: `row means Xb[labels == j].mean(axis=0)`.

Hint 3 · Almost there For the second part, pick the points of one cluster, define f as a small function (a lambda is fine), and compare $f(\text{mean})$ with f at 1000 perturbed points `m + rng.normal(scale=0.5, size=(1000, 2))`.

Mathematical insight · why f is minimised exactly at the mean

For any μ , writing $x_j - \mu = (x_j - \bar{x}) + (\bar{x} - \mu)$ and expanding,

$$\sum_j \|x_j - \mu\|^2 = \sum_j \|x_j - \bar{x}\|^2 + m \|\bar{x} - \mu\|^2,$$

because the cross term $2(\bar{x} - \mu)^\top \sum_j (x_j - \bar{x})$ is zero. The second term is ≥ 0 and vanishes

only at $\mu = \bar{x}$. This identity (Huygens' or König's theorem) also tells you *how much* a random point loses: m times its squared distance to the mean. Check it on your output.

Part III Linear Regression

Exercise 8 Least-squares line

Hint 2 · Method Every quantity is one NumPy line: `x.mean()`, `((x - xbar) ** 2).sum()`, `((x - xbar) * (y - ybar)).sum()`. Then $w = S_{xy}/S_{xx}$, $b = \bar{y} - w\bar{x}$.

Hint 3 · Almost there $R^2 = 1 - \sum e_i^2 / \sum (y_i - \bar{y})^2$. The CHECK compares with `np.polyfit`, which returns the slope *first*.

Exercise 9 Normal equations

Hint 2 · Method `X = np.column_stack([np.ones_like(x), x]);` then `np.linalg.solve(X.T @ X, X.T @ y)`. The result is (b, w) in that order, because the column of ones comes first.

Hint 3 · Almost there For the Ridge system, add `0.1 * np.eye(3)` to `X3.T @ X3`. (This toy version also penalises the intercept; that is fine for the question asked.)

Question 4 (written)

Hint 1 · Nudge Compute the ratio of the two coefficients of x and $2x$ in your output.

Hint 2 · Method All splits with $w_1 + 2w_2 = c$ give identical predictions. Minimise $w_1^2 + w_2^2$ under that constraint (a Lagrange multiplier, or draw the line $w_1 + 2w_2 = c$ and find its point closest to the origin).

Mathematical insight · why $X_3^\top X_3$ is singular

If $v = (0, 2, -1)^\top$, then $X_3 v = 2x - 2x = 0$. A non-zero vector in the null space of X_3 is also in the null space of $X_3^\top X_3$, so that matrix has a zero eigenvalue and determinant 0. Adding λI shifts every eigenvalue up by λ , so none is zero any more.

Exercise 10 Gradient descent

Hint 1 · Nudge Write the residual vector once, then both partial derivatives are means.

Hint 2 · Method $\partial L / \partial w = -2 \text{mean}(x_i r_i)$ and $\partial L / \partial b = -2 \text{mean}(r_i)$ with $r_i = y_i - wx_i - b$. The finite-difference check in the CHECK cell is how you would test any gradient you derive.

Hint 3 · Almost there For the 200-step runs, pass the feature as the argument `xx` to both `loss` and `grad`, so that the same code handles raw and centred x .

Question 5 (written)

Hint 1 · Nudge With centred x , what is $\sum_i (x_i - \bar{x})$? What does that do to $\partial L / \partial b$?

Hint 2 · Method Compute $X^\top X$ for the raw and the centred design matrices. Which one is diagonal?

Mathematical insight · finite differences

For a smooth function, $\frac{f(t+h)-f(t-h)}{2h} = f'(t) + O(h^2)$: the central difference is accurate to second order, much better than $\frac{f(t+h)-f(t)}{h}$, which is only first order. Too small an h is not better: rounding errors of size about $10^{-16}/h$ take over. $h \approx 10^{-6}$ is a good compromise for double precision.

Exercise 11 Ridge in 1-D

Hint 2 · Method Centre x and y inside the function, then apply $S_{xy} / (S_{xx} + \lambda)$. The intercept is $\bar{y} - w_\lambda \bar{x}$.

Hint 3 · Almost there The CHECK compares with `Ridge(alpha=lam)` from scikit-learn, which also leaves the intercept unpenalised.

Exercise 12 Outliers and leverage

Hint 1 · Nudge `np.append(x, 9)` adds a point; `np.polyfit(..., 1)[0]` is the slope.

Hint 2 · Method $H = X(X^\top X)^{-1}X^\top$; compute it as `Xd @ np.linalg.solve(Xd.T @ Xd, Xd.T)` and take `np.diag`.

Hint 3 · Almost there `HuberRegressor()` needs a 2-D input: `xb[:, None]`. The slope is `hub.coef_[0]`.

Question 6 (written)

Hint 1 · Nudge Which point has the largest h_{ii} ? Is its residual large or small after the fit?

Hint 2 · Method The trace of a matrix is the sum of its eigenvalues. What are the eigenvalues of a projection matrix, and how many are equal to 1?

Hint 3 · Almost there Huber reduces the influence of points with large residuals. Can a point that pulls the line towards itself have a large residual?

Mathematical insight · leverage for one feature

With an intercept and one feature, $h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{S_{xx}}$. It is smallest at $x_i = \bar{x}$ (where $h_{ii} = 1/n$) and grows quadratically with the distance to $\bar{x}$. Since $\hat{y} = Hy$, the fitted value at point i is $\hat{y}_i = h_{ii}y_i + \sum_{j \neq i} h_{ij}y_j$: a point with h_{ii} close to 1 almost fixes its own fitted value. That is

exactly what “leverage” means.